

Computer Science and Engineering Department, SVNIT, Surat
M.Tech. I – Semester I
High Performance Computing (CSDS125)
Lab Assignment 3
Parallel Linear Regression using OpenMP

1 Problem Statement

Implement a parallel version of multiple linear regression using OpenMP in C. The input features are two-dimensional (x_1, x_2) and the target output label y is one-dimensional, resulting in an overall 3-dimensional dataspace. Parallelization must be applied to the batch gradient descent computation, and its running time performance must be benchmarked and compared against its sequential counterpart using a large number of data points ($N = 10^7$).

2 Mathematical Formulation

Linear regression models the relationship between independent feature vectors $\mathbf{x}_i = [x_{i,1}, x_{i,2}]^T$ and a continuous dependent variable y_i .

2.1 Hypothesis Function

For a sample i , the predicted value $\hat{y}_i$ is given by:

$$\hat{y}_i = w_1 x_{i,1} + w_2 x_{i,2} + b \quad (1)$$

where w_1, w_2 are the feature weights and b is the bias (intercept).

2.2 Cost Function and Gradients

The Mean Squared Error (MSE) objective function minimized during training is:

$$J(w_1, w_2, b) = \frac{1}{2N} \sum_{i=0}^{N-1} (\hat{y}_i - y_i)^2 \quad (2)$$

The partial derivatives with respect to each model parameter are:

$$\frac{\partial J}{\partial w_1} = \frac{1}{N} \sum_{i=0}^{N-1} (\hat{y}_i - y_i) \cdot x_{i,1} \quad (3)$$

$$\frac{\partial J}{\partial w_2} = \frac{1}{N} \sum_{i=0}^{N-1} (\hat{y}_i - y_i) \cdot x_{i,2} \quad (4)$$

$$\frac{\partial J}{\partial b} = \frac{1}{N} \sum_{i=0}^{N-1} (\hat{y}_i - y_i) \quad (5)$$

2.3 Parameter Update Rules

At each epoch t , parameters are updated simultaneously using learning rate α :

$$w_1 \leftarrow w_1 - \alpha \frac{\partial J}{\partial w_1}, \quad w_2 \leftarrow w_2 - \alpha \frac{\partial J}{\partial w_2}, \quad b \leftarrow b - \alpha \frac{\partial J}{\partial b} \quad (6)$$

3 Parallelization Strategy with OpenMP

In batch gradient descent, the gradient computation involves summing errors across all N data points. Because $N = 10^7$, this loop represents the primary computational hotspot ($O(N)$ per epoch).

- **Data Decomposition:** The N iterations are statically split across available threads using `schedule(static)`. Since each iteration executes identical floating-point operations, static chunking ensures balanced workloads while introducing zero dynamic scheduling overhead.
- **Reduction Clause:** To eliminate race conditions without serialization locks, OpenMP's built-in reduction clause is applied:

```
#pragma omp parallel for reduction(+:dw1, dw2, db) schedule(static)
```

Each thread accumulates partial sums in private variables, which are automatically combined into the master accumulators at the end of the parallel region.

- **Memory Organization:** A Structure-of-Arrays (SoA) layout is adopted with separate continuous arrays for x_1 , x_2 , and y . This maximizes cache line reuse and enables SIMD vectorization.

4 C Implementation

The complete, self-contained C code using OpenMP is provided below:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <omp.h>
4
5 #define N 10000000 // 10 million datapoints (3D dataspace: x1, x2, y)
6 #define EPOCHS 20
7 #define LR 0.01
8
9 // Sequential Gradient Descent
10 void linear_regression_seq(double *x1, double *x2, double *y, int n, int epochs, double lr
11 , double *w1_out, double *w2_out, double *b_out) {
12     double w1 = 0.0, w2 = 0.0, b = 0.0;
13     for (int epoch = 0; epoch < epochs; epoch++) {
14         double dw1 = 0.0, dw2 = 0.0, db = 0.0;
15         for (int i = 0; i < n; i++) {
16             double pred = w1 * x1[i] + w2 * x2[i] + b;
17             double diff = pred - y[i];
18             dw1 += diff * x1[i];
19             dw2 += diff * x2[i];
20             db += diff;
21         }
22         w1 -= (lr / n) * dw1;
23         w2 -= (lr / n) * dw2;
24         b -= (lr / n) * db;
25     }
26     *w1_out = w1; *w2_out = w2; *b_out = b;
27 }
28
29 // Parallel Gradient Descent using OpenMP
30 void linear_regression_omp(double *x1, double *x2, double *y, int n, int epochs, double lr
31 , double *w1_out, double *w2_out, double *b_out) {
32     double w1 = 0.0, w2 = 0.0, b = 0.0;
33     for (int epoch = 0; epoch < epochs; epoch++) {
34         double dw1 = 0.0, dw2 = 0.0, db = 0.0;
35         #pragma omp parallel for reduction(+:dw1, dw2, db) schedule(static)
36         for (int i = 0; i < n; i++) {
37             double pred = w1 * x1[i] + w2 * x2[i] + b;
38             double diff = pred - y[i];
39             dw1 += diff * x1[i];
40             dw2 += diff * x2[i];
41             db += diff;
42         }
43         w1 -= (lr / n) * dw1;
44         w2 -= (lr / n) * dw2;
45         b -= (lr / n) * db;
46     }
47     *w1_out = w1; *w2_out = w2; *b_out = b;
48 }
49
50 int main() {
51     printf("=== Linear Regression with OpenMP ===\n");
52     printf("Datapoints (N): %d | Epochs: %d | Learning Rate: %.2f\n", N, EPOCHS, LR);
53
54     // Allocate 3-dimensional dataspace: (x1, x2) features and y label
55     double *x1 = (double *)malloc(N * sizeof(double));
56     double *x2 = (double *)malloc(N * sizeof(double));
```

```
55     double *y = (double *)malloc(N * sizeof(double));
56
57     if (!x1 || !x2 || !y) {
58         fprintf(stderr, "Memory allocation failed!\n");
59         return 1;
60     }
61
62     // Generate synthetic data (True weights: w1=2.0, w2=3.0, b=1.0)
63     #pragma omp parallel for schedule(static)
64     for (int i = 0; i < N; i++) {
65         x1[i] = (double)(i % 100) / 100.0;
66         x2[i] = (double)((i * 3) % 100) / 100.0;
67         y[i] = 2.0 * x1[i] + 3.0 * x2[i] + 1.0;
68     }
69
70     double w1_seq = 0, w2_seq = 0, b_seq = 0;
71     double w1_par = 0, w2_par = 0, b_par = 0;
72
73     // 1. Sequential Run
74     double t_seq_start = omp_get_wtime();
75     linear_regression_seq(x1, x2, y, N, EPOCHS, LR, &w1_seq, &w2_seq, &b_seq);
76     double t_seq = omp_get_wtime() - t_seq_start;
77
78     // 2. Parallel Run
79     double t_par_start = omp_get_wtime();
80     linear_regression_omp(x1, x2, y, N, EPOCHS, LR, &w1_par, &w2_par, &b_par);
81     double t_par = omp_get_wtime() - t_par_start;
82
83     // Results & Performance Comparison
84     printf("\n--- Sequential Performance ---\n");
85     printf("Time: %.4f seconds\n", t_seq);
86     printf("Learned Parameters: w1 = %.4f, w2 = %.4f, b = %.4f\n", w1_seq, w2_seq, b_seq);
87
88     printf("\n--- OpenMP Parallel Performance (%d threads) ---\n", omp_get_max_threads());
89     printf("Time: %.4f seconds\n", t_par);
90     printf("Learned Parameters: w1 = %.4f, w2 = %.4f, b = %.4f\n", w1_par, w2_par, b_par);
91
92     printf("\n--- Summary ---\n");
93     printf("Speedup: %.2fx\n", t_seq / t_par);
94
95     free(x1);
96     free(x2);
97     free(y);
98     return 0;
99 }
```

5 Experimental Results and Performance Comparison

The program was compiled with `gcc -O3 -fopenmp linear_regression_omp.c -o linear_regression -lm` and executed on an x86_64 multi-core platform with $N = 10,000,000$ points and 20 epochs.

5.1 Terminal Output

```
=== Linear Regression with OpenMP ===
Datapoints (N): 10000000 | Epochs: 20 | Learning Rate: 0.01

--- Sequential Performance ---
Time: 0.3554 seconds
Learned Parameters: w1 = 0.3430, w2 = 0.3542, b = 0.6003

--- OpenMP Parallel Performance (4 threads) ---
Time: 0.2083 seconds
Learned Parameters: w1 = 0.3430, w2 = 0.3542, b = 0.6003

--- Summary ---
Speedup: 1.71x
```

5.2 Multi-Thread Scalability Comparison

The execution times and speedups across varying numbers of OpenMP threads are summarized in Table 1.

Configuration	Thread Count (P)	Execution Time (s)	Speedup (S)	Efficiency (E)
Sequential Baseline	1	0.3554	1.00×	100.0%
OpenMP Parallel	1	0.2904	1.22×	122.0%
OpenMP Parallel	2	0.2109	1.69×	84.5%
OpenMP Parallel	4	0.2083	1.71×	42.8%

Table 1: Running time and speedup comparison for $N = 10^7$ datapoints over 20 epochs.

6 Analysis and Discussion

- Convergence and Accuracy:** Both sequential and OpenMP parallel gradient descent converged to identical parameter values ($w_1 = 0.3430$, $w_2 = 0.3542$, $b = 0.6003$). This confirms that the reduction clause preserves numerical correctness across threads.
- Speedup Evaluation:** The OpenMP parallel execution achieves a speedup of 1.71× over the sequential counterpart on 4 threads. Moving from 1 to 2 threads yields substantial gains (time drops from 0.3554 s to 0.2109 s).
- Memory Bandwidth Bottleneck:** While compute scales linearly with threads, batch gradient descent on large arrays ($N = 10^7$, consuming ≈ 240 MB of memory) is memory-bound. Multiple threads contend for shared memory bus bandwidth to stream continuous

float vectors from RAM to CPU cache, explaining the sub-linear scaling at higher thread counts in accordance with Amdahl's Law.

4. **Granularity Conclusion:** By parallelizing the outer loop across 10^7 iterations and using static scheduling, thread creation and synchronization overhead is amortized, demonstrating clear performance benefits for large-scale data modeling.